Deterministic Ramanujan graphs and SpiderCat at every fault weight

Feasibility of OpenAI's construction, a certified practical substitute, and an audit of the SpiderCat data

Technical report for the SpiderCat project

8 October 2026

Abstract

We asked whether OpenAI's preprint *Deterministic nonbipartite Ramanujan graphs in every fixed degree* can be used to generate SpiderCat marked graphs that are fault tolerant at every fault weight ($t = \infty$) for prescribed CAT sizes n. It cannot. No implementation exists, the algorithm is not fully specified, and its own parameter choices are meaningful only for starting graphs with $\log_{10} N \gtrsim 10^6$.

It is also not needed. SpiderCat's $t = \infty$ argument uses only the one-sided bound $\lambda_2(H) \leq d - 2k$ on the starting graph H. Random regular graphs meet this bound with constant probability (within two samples in all our runs). OpenAI's own final test, made one-sided, certifies it.

We prove a composition lemma that turns any such starting graph and robust gadgets into a $t = \infty$ marked graph. With it we generated and certified graphs for every n tested, from 24 to 20 000: vertex ratio 12 for even n, generation $\leq$3.8 s, independent certification $\leq$7.6 s. For SpiderCat's own sizes, a small dense starting graph with exactly computed expansion lowers the vertex ratio to 4.0–7.6 ($n = 24$–100). Its certificates are fully exact, and its circuits use $V + n + 1$ CNOTs, the minimum for the given graph.

An exact audit of the repository found four problems:

- The 26 stored "$t = 7$" graphs ($n = 24$, 25, 27–50) are only 4-robust.
- 8 of their circuits, and all 10 stored $t = \infty$ circuits, do not prepare the CAT state.
- The $t = \infty$ branch fails for every tested n from 20 to 67.
- The verifier `find_marking_property_violation` is unsound.

Contents

1 Setting: marked graphs and the all-weight condition **2**

2 What OpenAI's construction provides, and how it enters SpiderCat **2**

3 Can OpenAI's construction be run? **4**

4 A practical certified pipeline **5**

4.1 Ramanujan route 5
4.2 Dense route 5
4.3 Independent certificate 6
4.4 End-to-end exact check 7
4.5 Circuits 7

5 Results **7**

6 Validation **8**

7 Findings about the SpiderCat repository and paper **9**
7.1 The stored $t = 7$ graphs are only 4-robust 9
7.2 Stored circuits that do not prepare the CAT state 11
7.3 The $t = \infty$ branch of `generate.py` 11
7.4 The verifier is unsound 11
7.5 Corrections to the paper 12

8 Recommendations **12**

A Reproducibility **13**

1 Setting: marked graphs and the all-weight condition

A SpiderCat *marked graph* (G, M) is a cubic graph G whose edges carry marks. Each mark is one CAT output: a degree-2 boundary spider subdividing its edge. Let G' be G with every mark turned into a degree-2 vertex on its edge. Write $m(S)$ for the number of mark vertices in $S \subseteq V(G')$ and n for the total number of marks.

Definition 1 (robust at every weight)**.** (G, M) is *robust at every weight* ($t = \infty$) if

$$\min\big(m(S),\, n - m(S)\big) \;\le\; |\delta_{G'}(S)| \qquad \text{for every } S \subseteq V(G').$$

This is SpiderCat's treatment of cuts (Definitions 2.2–2.4, Propositions 2.2 and 5.2–5.3, Appendices B and C.5 of the SpiderCat paper [1]):

- A cut may be placed anywhere along an edge, never at a vertex or a mark.
- Undetectable X faults are exactly the coboundaries $\delta(S)$ for arbitrary S (proof of Proposition 5.2), so Definition 1 quantifies over all S, connected or not. This agrees with the paper's two-component cuts: whenever some S violates the inequality, so does a cut whose two sides are both connected.
- Pushed to the outputs, the fault $\delta(S)$ has weight $\min(m(S), n - m(S))$ modulo the stabiliser $X^{\otimes n}$.
- Z and Y components add at most one output Z, which can sit on an output already carrying the X part.
- A fault on an output wire raises the fault weight by one and the output weight by at most one, so such faults never matter.

Two remarks.

- Definition 5.3 of [1] says a cut is good if one side has "at most t marks". The surrounding text, the proofs and the code all use "at most w marks" for a cut of w edges, which is the reading above. Read literally, the printed version is vacuous at $t = \infty$.
- Since $\min(m, n - m) \le \lfloor n/2 \rfloor$, robustness at every weight is equivalent to t-robustness with $t = \lfloor n/2 \rfloor - 1$. SpiderCat's stored and SAT-generated graphs (with $t \le 7$) therefore reach $t = \infty$ only for $n \le 17$.

2 What OpenAI's construction provides, and how it enters SpiderCat

The theorem. OpenAI's preprint [2] (main theorem, `introduction.tex`) states: for every $d \ge 3$ there are constants $n_0(d)$, k_d and a deterministic algorithm. On every even input $N \ge n_0(d)$,

it outputs a simple d-regular graph on N vertices in $O_d(N^{k_d})$ bit operations, with $-2\sqrt{d-1} < \lambda < 2\sqrt{d-1}$ for every eigenvalue of the adjacency matrix restricted to $\mathbf{1}^\perp$.

Composition. SpiderCat's Theorem C.6 replaces every vertex of a regular expander H by a fault-equivalent decomposition of a CAT state. We state it combinatorially, directly in the cut model of Definition 1.

Definition 2 (composed graph)**.** Let H be a loopless multigraph and $k\colon V(H) \to \mathbb{Z}_{\geq 0}$ an output count. A *gadget* for v is a cubic marked graph Γ_v whose marks (*terminals*) number $T_v \geq \deg_H(v) + k_v$. The composed graph F is built as follows:

1. Take the disjoint union of the gadgets.
2. For every H-edge $\{u, v\}$, join a fresh terminal of Γ_u to a fresh terminal of Γ_v. These terminals become degree-3 *ports*.
3. In each gadget, keep k_v further terminals as marks (the outputs) and delete the rest.

We write Γ'_v and F' for Γ_v and F with every terminal or mark subdivided as in G'.

Lemma 3 (composition)**.** *Assume that every gadget is robust at every weight with respect to its terminals, and that*

$$|\delta_H(R)| \;\geq\; \min\bigl(k(R), k(V(H) \setminus R)\bigr) \qquad \text{for all } R \subseteq V(H), \tag{1}$$

where $k(R) = \sum_{v \in R} k_v$. Then F is a cubic marked graph with $n = \sum_v k_v$ marks that is robust at every weight.

Proof. Deleting terminals. Let a gadget have T terminals, of which a set A is kept, and let S be a vertex set of the reduced gadget (with A subdivided). Put each deleted terminal vertex on the side of one of its neighbours; this leaves $|\delta(S)|$ unchanged. Let $a = |A \cap S|$, and let $b \leq T - |A|$ be the number of deleted terminals placed in S. Robustness of the full gadget gives $\min(a+b,\, T-a-b) \leq |\delta(S)|$, and $\min(a, |A|-a) \leq \min(a+b,\, T-a-b)$. So the reduced gadget is robust, and we may assume $T_v = \deg_H(v) + k_v$.

Main argument. Fix $S \subseteq V(F')$. For each v:

- let w_v be the number of cut F'-edges inside Γ'_v;
- let τ_v be the number of terminals of Γ_v (ports and outputs) lying in S;
- let $\sigma(v) = 1$ if the majority of v's terminals lie in S ($\tau_v > T_v/2$), $\sigma(v) = 0$ if $\tau_v < T_v/2$, and either value on a tie.

The terminals on the side opposite to $\sigma(v)$ are *minority* terminals. Their number is $\min(\tau_v, T_v - \tau_v) \leq w_v$, by robustness of Γ_v applied to $S \cap V(\Gamma'_v)$. Split this number as $\mathrm{mp}(v) + \mathrm{mo}(v)$ (minority ports and minority outputs), and put $R = \{v : \sigma(v) = 1\}$.

Consider an H-edge $\{u, v\}$ with $\sigma(u) \neq \sigma(v)$. Either its port–port edge is cut, or its two ports lie on the same side of S; then one of them is a minority port of its own gadget. Every port belongs to exactly one H-edge, so

$$|\delta_H(R)| \leq c_{\mathrm{port}} + \sum_v \mathrm{mp}(v),$$

where c_{port} counts the cut port–port edges.

For the outputs, $m(S) \leq k(R) + \sum_{v \notin R} \mathrm{mo}(v)$ and $n - m(S) \leq k(V \setminus R) + \sum_{v \in R} \mathrm{mo}(v)$. Hence

$$\begin{aligned} \min(m(S), n - m(S)) &\leq \min\bigl(k(R), k(V \setminus R)\bigr) + \textstyle\sum_v \mathrm{mo}(v) \\ &\leq |\delta_H(R)| + \textstyle\sum_v \mathrm{mo}(v) \;\leq\; c_{\mathrm{port}} + \textstyle\sum_v w_v \;=\; |\delta_{F'}(S)|. \end{aligned}$$

The last equality holds because the edges of F' are exactly the gadget edges and the port–port edges. □

Lemma 4 (spectral certificate)**.** *Let H be d-regular on N vertices, with $k_v \le k$ for all v. If $\mu_2 = d - \lambda_2(A_H) \ge 2k$, then* (1) *holds.*

Proof. Evaluating the Laplacian quadratic form at $\mathbf{1}_R - \frac{|R|}{N}\mathbf{1}$ gives $|\delta_H(R)| \ge \mu_2 |R||V \setminus R|/N \ge \frac{\mu_2}{2}\min(|R|, N - |R|)$, because $\max(|R|, N - |R|) \ge N/2$. Moreover $\min(k(R), k(V \setminus R)) \le k \min(|R|, N - |R|)$. □

Instantiation with OpenAI's graphs. Take $d = 10$ and $k = 2$. Each starting vertex becomes the stored 12-terminal gadget `cat_state_t5_n12` (the Heawood graph on 14 vertices with 12 marked edges). Ten of its terminals become ports and two stay outputs. The relevant quantities are:

starting-graph vertices	N (even, $N \ge n_0(10)$; H is 10-regular)
outputs (marks)	$n = 2N$ (other n: round N up and drop marks)
final internal (degree-3) vertices	$V = N(14 + 10) = 24N$, i.e. $12n$ when $n = 2N$

Lemma 4 needs $\lambda_2 \le 6 = 2\sqrt{9}$. That is exactly the Ramanujan threshold, with zero slack, and it is *one-sided*: the nonbipartite part of OpenAI's result, which controls the lower spectral edge, contributes nothing.

In principle, OpenAI's graphs would replace the "Ramanujan (all n), $\le 5 \times 10^6 n$" row of SpiderCat's Table 1 by about $13n + 1$ CNOTs (Procedure 4.1) or $37.4n$ CNOTs (depth 3), for $n \ge 2n_0$. For a one-sided bound, the bipartite Ramanujan multigraphs of every even order of Marcus, Spielman and Srivastava [3], made deterministic in polynomial time by Cohen [4], already suffice for this purpose. Both are cited in OpenAI's own introduction. Multigraphs are admissible in Lemma 3.

3 Can OpenAI's construction be run?

Availability. The repository folder contains only `paper.pdf`, the LaTeX sources and a README. There is no code and no pseudocode beyond the proofs. The collection catalogue (family 178) and `lean/formalization.yaml` have no Lean entry for this paper. Neither $n_0(d)$ nor k_d is ever made explicit; dozens of constants and exponents are only "sufficiently small/large".

Size threshold. Here N is the order of the constructed graph (the preprint calls it n; the CAT size $2N$ differs negligibly on this scale). The paper's own parameter table (`setup.tex`, with $a = 10^{-4}$, $q = d - 1$) fixes several logarithmic radii and stopping sizes. They become meaningful only for astronomically large N (Table 1, $d = 10$). At $N = 10^6$ every one of these radii rounds to 0. An independent full reading found further requirements of the same kind, up to $\log_{10} N \approx 10^7$.

Table 1: Approximate lower bounds on the order N of the constructed graph implied by the parameter choices of [2] ($d = 10$, $q = 9$, $a = 10^{-4}$). The scales hold up to the fixed factors and integer rounding that the preprint's "$\asymp$" allows, so these are order-of-magnitude thresholds.

requirement	scale in `setup.tex`	approximate threshold
girth-exclusion radius $g_0 \ge 1$	$q^{g_0} = N^{0.002a}$	$\log_{10} N \gtrsim 4.77 \times 10^6$
final matching $l_3 = N^{\chi} \ge 2$, $\chi < r_P = 0.001a$	$l_3 = N^{\chi}$	$\log_{10} N \gtrsim 3.01 \times 10^6$
boundary separation radius $R_0 \ge 1$	$q^{R_0} = N^{0.04a}$	$\log_{10} N \gtrsim 2.39 \times 10^5$

Cost. Even formally, the polynomial is out of reach:

- Early pairing and cleanup track 3 333 334 auxiliary spectral parameters at the largest primary h_*, with both signs. That is about 6.7 million dense $N \times N$ precision matrices, updated over $\Theta(N)$ pairing steps with $O(N^2)$ candidate pairs each.

- The selection rule uses moment accounts of order $p > 4 \times 10^6$ and a colour-coding dynamic programme with at least N^{24} states.
- The repair stage enumerates, in the worst case, a sample space of $P^2 \geq N^{2C'}$ prime-field seeds (a $1 - o(1)$ fraction of seeds succeed), and evaluates Bernstein polynomials of degree $\gtrsim N^5$.

An independent reading estimates the exponent at $k_{10} \gtrsim 30$.

What is reusable. Only the final exact test (`algorithm.tex`): positive definiteness of $B^{\mathsf{T}}(4qI - A^2)B$, which certifies *any* candidate graph. SpiderCat needs only its one-sided analogue, $6I - A + J \succ 0$. For 10-regular A and $N > 4$ this is equivalent to $\lambda_i < 6$ on $\mathbf{1}^{\perp}$. We certify it rigorously in floating point:

- Run a Cholesky factorisation of $M - \varepsilon I$, with ε a power of two.
- Apply Higham's backward-error bound $|\Delta M| \leq \gamma_{N+1}|L||L^{\mathsf{T}}|$ [5]. It gives $\lambda_{\min}(M) \geq \varepsilon - \gamma_{N+1}\|L\|_F^2$.

The bound holds in IEEE binary64 for any ordering of the inner products, so also for LAPACK's blocked Cholesky, assuming no underflow. $M - \varepsilon I$ is formed exactly, and the computed bound is inflated by 1% to cover its own rounding.

In all our runs the error bound stayed at or below 7.9×10^{-8}, compared with $\varepsilon \approx 10^{-3}$. For the benchmark instances with $N \in \{12, 25, 40, 50, 100\}$, exact fraction-free (Bareiss) elimination agreed.

Verdict. No useful instance can be generated from the construction. Any candidate graph can be certified in $O(N^3)$, which takes seconds up to $N = 10^4$.

4 A practical certified pipeline

All code is in the new module `spidercat/graphs_ramanujan.py`. It reuses the repository's stored graphs (as gadgets), `load_solution_triplet` and `CatStateExtractor`.

4.1 Ramanujan route

Sample a random simple 10-regular graph (networkx; asymptotically uniform) on $N = \max(\lceil n/2 \rceil, 11)$ vertices, and accept it once $6I - A + J \succ 0$ is certified. The vertex ratio is 12 for even $n \geq 22$ and $12 + 12/n$ for odd n.

In our benchmarks at most 2 samples were needed (Table 2). The threshold has zero slack, however, so asymptotically a constant fraction of samples fails. Moreover, the gap $6 - \lambda_2$ shrinks like $N^{-2/3}$; together with the $O(N^3)$ cost, this limits the floating-point certificate to N of order 10^5.

A variant with slack is $d = 11$, $k = 2$ with the stored 13-terminal gadget. It needs only $\lambda_2 \leq 7$, against $2\sqrt{10} \approx 6.32$, at vertex ratio 12.5.

4.2 Dense route

Lemma 3 holds for any loopless H satisfying (1). For small N, two choices of H give exact expansion:

- $H = K_N$, with $|\delta(R)| = |R|(N - |R|) \geq \lceil N/2 \rceil \min(|R|, N - |R|)$, so $k_v \leq \lceil N/2 \rceil$ is allowed;
- a random d-regular H whose exact Cheeger constant h is computed by enumeration ($N \leq 22$), giving $k = \lfloor h \rfloor$.

Table 2: Second adjacency eigenvalue of random simple 10-regular graphs (fixed seed).

N	samples	$\lambda_2 < 6$	median λ_2	max λ_2
12	50	50/50	0.000	0.000
25	50	50/50	3.894	4.579
50	50	50/50	5.055	5.448
100	50	50/50	5.496	5.758
250	40	40/40	5.772	5.934
500	30	29/30	5.901	6.001
1 000	20	19/20	5.941	6.006
2 000	10	10/10	5.962	5.991

A vertex then costs $V_\Gamma(\deg + k_v) + \deg$ internal vertices, where $V_\Gamma(T)$ is the size of the smallest certified gadget with at least T terminals. Per output this is much cheaper than the Ramanujan route, whose 10 ports serve only 2 outputs. A small knapsack chooses N, d and the k_v.

Table 3 lists the gadget library:

- SpiderCat's stored graphs for 7–13 terminals;
- four new gadgets for 14–17 terminals, found by local search with an exact objective (shipped in `spidercat/tinf_gadgets/`).

The new gadgets are themselves standalone $t = \infty$ graphs for $n = 14$–17, at vertex ratio 1.33–1.53; the repository has no stored graphs at $t = 6$ for $n = 14$–20 or at $t = 7$ for $n = 14$–23. Table 4 gives the resulting plans.

Table 3: Gadget library. Each gadget is certified by exhaustive enumeration of all vertex subsets with the exact mark-splitting formula, by an exact MILP (optimum 0), and, when $|V(G')| \leq 30$, by enumeration of every subset of $V(G')$. Times in brackets.

terminals	V	girth	marks/edge	source	exhaustive	MILP	full G'
7	4	3	2	stored	0 (0.3 ms)	0 (6.3 ms)	0 (0.1 ms)
8	6	4	1	stored	0 (0.1 ms)	0 (10 ms)	0 (0.2 ms)
9	6	4	1	stored	0 (0.1 ms)	0 (8.1 ms)	0 (0.3 ms)
10	10	5	1	stored	0 (0.1 ms)	0 (24 ms)	0 (15 ms)
11	10	5	1	stored	0 (0.1 ms)	0 (36 ms)	0 (31 ms)
12	14	6	1	stored	0 (1.0 ms)	0 (81 ms)	0 (1.0 s)
13	14	6	1	stored	0 (0.7 ms)	0 (99 ms)	0 (2.0 s)
14	20	6	1	new	0 (21 ms)	0 (168 ms)	–
15	20	5	1	new	0 (22 ms)	0 (197 ms)	–
16	24	6	1	new	0 (341 ms)	0 (282 ms)	–
17	26	6	1	new	0 (1.4 s)	0 (423 ms)	–

4.3 Independent certificate

`verify_composition_certificate` receives only the graph F, its marks and the partition of $V(F)$ into clusters. It checks:

1. F is simple, cubic and connected, with at most two marks per edge, and no mark lies on an edge between clusters.
2. Every cluster vertex with an outside neighbour has exactly one, and every boundary vertex lies on a path between internal vertices.
3. Each cluster, read as a gadget whose terminals are its ports and its output marks, is robust: isomorphic clusters are certified once, by exhaustive enumeration.

Table 4: Cheapest dense-route plans found by the planner, compared with the Ramanujan route, whose size is $V = 24\max(\lceil n/2 \rceil, 11)$.

n	starting graph H	N	degree	outputs/vertex	V	V/n	V (Ramanujan)
18	K_6	6	5	3	66	3.67	264
24	K_7	7	6	3–4	96	4.00	288
32	K_8	8	7	4	136	4.25	384
45	K_9	9	8	5	198	4.40	552
50	K_{10}	10	9	5	290	5.80	600
64	K_{11}	11	10	5–6	366	5.72	768
72	K_{12}	12	11	6	444	6.17	864
80	random 11-regular	16	11	5	560	7.00	960
100	random 12-regular	20	12	5	760	7.60	1 200

4. The quotient multigraph satisfies (1): by enumeration for up to 20 clusters, otherwise by the rigorous spectral test of Lemma 4.

Lemma 3 then yields robustness at every weight.

4.4 End-to-end exact check

`max_margin_milp` maximises $m(S) - |\delta_{F'}(S)|$ subject to $m(S) \le \lfloor n/2 \rfloor$ with HiGHS. Robustness holds if and only if the optimum is 0. This check does not use Lemma 3. With an extra constraint $|\delta(S)| \le t$, it decides t-robustness; we used that form for the repository audit in Section 7.

4.5 Circuits

The repository extractor handles a non-tree edge correctly only if it carries a mark. It turns an unmarked co-tree edge into a flag node and hangs it on the spanning tree as a leaf. A qubit line then ends on that flag instead of an output, data-qubit indices overflow into the flag range, and the circuit prepares a GHZ state on more than the n data qubits.

A connected cubic graph with V vertices has $V/2+1$ co-tree edges, so this failure is unavoidable once $V/2 + 1 > n$ (that is, $V \ge 2n - 1$). That covers every $t = \infty$ graph: on the Ramanujan route $V/2 + 1 = 6n + 1$, and the prime-inverse branch has $V = 2n$. It also happens whenever a chosen spanning tree leaves an unmarked co-tree edge; Section 7.2 gives stored examples.

We therefore make two changes:

- build a spider-ordering tree (Definition 4.2) with a MILP: a spanning tree with single-commodity flow constraints, in which every leaf is matched to a distinct mark on a non-tree edge;
- keep unmarked co-tree edges unsubdivided, so each becomes one post-selected ancilla with two CNOTs, as in Procedure 4.1.

`circuit_fault_check` propagates Pauli frames for all single $X/Y/Z$ faults on every wire segment of the circuit. It then enumerates every undetected fault set of weight at most 2 and requires output weight $\le$ fault weight modulo the CAT stabilisers. Its controls behave as expected: a plain CNOT chain and SpiderCat's own $t = 1$ circuit fail, while the stored $t = 3$ and $t = 5$ circuits pass.

5 Results

All timings are wall-clock on an Apple M3 Pro (11 cores, 36 GB). Other jobs were running at the same time, so the timings are upper bounds.

Table 5: Ramanujan route. "generate" covers rejection sampling, the rigorous certificate of H and the composition. "certify" is the independent re-check from $(F, \text{marks}, \text{partition})$. "two-sided" reports OpenAI's own strict test on the sampled H (for information only).

n	N	V	V/n	tries	$\lambda_2(H)$	generate	certify	two-sided
24	12	288	12	1	0.000	1.8 ms	8.7 ms	yes
50	25	600	12	1	4.443	2.2 ms	22 ms	yes
80	40	960	12	1	4.382	5.2 ms	28 ms	yes
100	50	1 200	12	1	4.756	6.7 ms	69 ms	yes
200	100	2 400	12	1	5.429	13 ms	124 ms	yes
500	250	6 000	12	1	5.765	34 ms	209 ms	yes
1 000	500	12 000	12	1	5.839	55 ms	514 ms	yes
2 000	1 000	24 000	12	2	5.887	279 ms	823 ms	yes
5 000	2 500	60 000	12	1	5.991	547 ms	2.4 s	yes
10 000	5 000	120 000	12	1	5.986	1.8 s	2.8 s	yes
20 000	10 000	240 000	12	1	–	3.8 s	7.6 s	yes

Table 6: Dense route at SpiderCat's example sizes. Certificates are fully exact (exhaustive gadgets and exhaustive quotient). Circuits come from the MILP spider-ordering tree; the CNOT count equals $V + n + 1$ (Proposition 4.1) in every case. "circuit": noiseless output is the CAT state and every undetected fault set of weight ≤ 2 is harmless. For $n = 100$, generation and certification come from the main run. Its circuit comes from a re-run with the final tree MILP (2.6 min), because the earlier formulation used in the main run, which kept marked edges out of the tree, found no tree for this graph.

n	starting graph	N	V	V/n	generate	certify	CNOTs	qubits (flags)	circuit
24	K_7	7	96	4.0	3.5 ms	2.1 ms	121	73 (49)	pass
50	K_{10}	10	290	5.8	968 ms	32 ms	341	196 (146)	pass
80	11-regular, $h = 5$	16	560	7.0	1.8 s	461 ms	641	361 (281)	pass
100	12-regular, $h = 5$	20	760	7.6	1.4 s	2.0 s	861	481 (381)	pass

6 Validation

Every claim above was checked independently, in most cases by separate reviewers with their own code.

- **Lemma 3.**
  - Three independent reviews found no gap: proof, semantics and the spectral step.
  - One reviewer checked about 11,600 random compositions by exhaustive enumeration over $V(F')$ (9,500 random and 2,100 tight instances, including 2-terminal gadget edges and parallel H-edges), with no violation; it also exhaustively tested 59,972 gadget terminal placements. Negative controls (non-expanding H or a non-robust gadget) produced violations.
  - A second reviewer ran 156 further compositions with no violation; its 50 non-expanding controls all failed.
- **Exact checkers.** An independent brute force over all subsets of $V(G')$ agreed with `max_margin_exhaustive`, the full-G' enumerator and the MILP (reported as about 540 random marked graphs, including disconnected cuts; 0 disagreements). On 93 further graphs with 3–4 marks on an edge, the full-G' enumerator and the MILP agreed, and the vertex-subset formula correctly refuses them; it would be wrong for them.
- **Certificate verifier.**
  - Fuzzing (1,600 structured compositions and 300 mutated graphs) produced no false certificate among the $107 + 89$ certified instances that could be checked exhaustively.

Table 7: Circuits for the Ramanujan route. The tree MILP is the bottleneck for $V \gtrsim 900$. The dense-route tree MILP for $V = 760$ took 2.6 min, with 20 marked edges inside the tree.

n	V	CNOTs ($= V + n + 1$)	qubits (flags)	tree MILP	circuit
24	288	313	169 (145)	11.4 s	pass
50	600	651	351 (301)	2.4 min	pass
80	960	tree MILP stopped at its 10.0 min limit; no circuit			
100	1 200	tree MILP stopped at its 10.0 min limit; no circuit			

Table 8: End-to-end exact certification of composed graphs by MILP over all $S \subseteq V(F')$; this is independent of Lemma 3. The Ramanujan $n = 50$ instance and the dense $n = 24, 50$ instances are exactly those of Tables 5–7. The others were generated by the same code: Ramanujan $n = 24$ in an earlier run (its H, the complement of a perfect matching in K_{12}, is unique up to isomorphism), Ramanujan $n = 25$, and the dense $n = 18, 32, 45$ plans of Table 4.

route	n	$\lvert V(F')\rvert$	time	result
Ramanujan	24	312	5.4 s	robust (optimum 0)
Ramanujan	25	337	7.3 s	robust (optimum 0)
Ramanujan	50	650	40.5 min	robust (optimum 0)
dense (K_N)	18	84	3.6 s	robust (optimum 0)
dense (K_N)	24	120	8.8 s	robust (optimum 0)
dense (K_N)	32	168	29.2 s	robust (optimum 0)
dense (K_N)	45	243	12.6 min	robust (optimum 0)
dense (K_N)	50	340	25.7 min	robust (optimum 0)

  - Two soundness gaps were found in an earlier version (marks on port–port edges; boundary-only cycles inside a cluster), each with a concrete counterexample. Both are closed in the current version.
  - The floating-point positive-definiteness certificate gave no false positive on 300 near-singular integer matrices. It is not certified for rook graphs with λ_2 exactly equal to the bound, matching exact Bareiss elimination.

- **Circuits.**
  - A reviewer found that the first version of `circuit_fault_check` placed faults per stim instruction rather than per gate, so faults between merged CNOTs were missed. This is fixed.
  - The reviewer generated six circuits of its own (dense $n = 18, 24, 32$; Ramanujan $n = 18, 24, 32$) and re-checked them per gate with independent stim-based code at weight ≤ 2; all six passed.
  - According to its report, four of them (dense $n = 18, 24, 32$ and Ramanujan $n = 18$) also passed an exhaustive weight-3 check, with up to 2.6×10^8 undetected triples enumerated. Two of these four runs are in its stored logs.
  - The circuits in Tables 6 and 7 were checked with the corrected `circuit_fault_check`.

7 Findings about the SpiderCat repository and paper

7.1 The stored $t = 7$ graphs are only 4-robust

We checked all 437 stored graphs (`circuits_data`) exactly at their effective $t = \min(t, \lfloor n/2 \rfloor - 1)$, using the cut-bounded MILP. 411 are robust. All 26 failures are the $t = 7$ graphs with $n = 24$, 25, 27–50; no $t = 7$ graph is stored for $n = 26$ (Table 9). Every one of them is exactly robust up to cut weight 4 and violated at weight 5.

Table 9: Exact audit of the stored graphs at their effective t.

t	files	n range	robust	violated
2	94	7–100	94	0
3	94	7–100	94	0
4	87	7–100	87	0
5	94	7–100	94	0
6	36	7–50	36	0
7	32	7–50	6	26

Witnesses ($n = 24$).

- *Weight 5.* $S = \{21, 22, 23\}$ is a path. Its 7 incident edges carry 6 marks behind a cut of 5, so a weight-5 fault spreads to output weight 6.
- *Weight 7.* $S = \{14, 15, 21, 22, 23\}$ induces a tree with boundary edges (1,22), (6,23), (8,15), (13,14), (15,16), (20,21), (23,24). Its 4 internal edges carry 4 marks and its boundary edges 5. With the cut placed beyond those marks, $m(S) = 9$ and $n - m(S) = 15$, so a weight-7 fault spreads to output weight 9.

Circuit-level confirmation on the stored `.stim` files.

- An exact MILP over all wire-segment Pauli faults searches for the worst undetected fault of weight $\leq w$ (Table 10).
- The weight-7 fault it finds (7 single-qubit X errors at (gate index, qubit) = (init,7), (7,1), (18,5), (19,9), (32,13), (46,17), (68,29)) was also replayed in stim's tableau simulator [7]. It triggered none of the 16 flag measurements and left an output X error of weight 9 modulo the CAT stabilisers.

Table 10: Worst undetected circuit faults of weight $\leq w$ (exact MILP over wire-segment faults).

circuit	w	worst output weight	verdict	time
`t5_n24`	5	5	within w	2.2 s
`t6_n24`	6	6	within w	23.4 s
`t7_n24`	4	4	within w	2.6 s
`t7_n24`	5	6	**exceeds** w	8.0 s
`t7_n24`	6	7	**exceeds** w	6.1 s
`t7_n24`	7	9	**exceeds** w	1.1 min

Cause.

- `GraphMarker.find_solution(7)` (`markings.py`, line 412) imposes only the total-dominating-set constraint. Appendix E.2 of [1] gives that constraint as the encoding for $T = 7$ (and Section 6 calls finding $t = 7$ markings "equivalent" to finding a total dominating set). It adds none of the subtree constraints that the same appendix introduces as necessary conditions.
- The stored graphs satisfy the total-dominating-set constraint but violate the subtree conditions (the weight-5 witness above).
- `general_solver` uses the correct bound $|S|+2$, and for $T = 7$ it would reject these markings, since it covers subtrees of up to 3 vertices. It is still incomplete: its loop `range(1, T - 1)` (line 407) never imposes the 4- and 5-vertex conditions. It should run over `range(3, T + 1)`.

- Verification was off. `cat_state_FT` defaults to `run_verification=False`, and `process_cell` passes `False`. Switched on, even the existing verifier flags all 26 graphs (for $n = 24$ it returns $\{21, 22, 23\}$ in 5 ms).
- The local conditions implied by Definition 5.3 are: for every subtree S with $|S| \leq t - 2$, the edges touching S carry at most $|S| + 2$ marks. Appendix E.2 writes "$\leq t$", the same t-versus-w slip as in Definition 5.3. These conditions are necessary; robustness also requires the absence of small nonlocal cuts.

Impact. This affects the claim "$n \leq 50$ and $t \leq 7$" and the $t = 7$ simulation data. Re-marking the stored $n = 24$ graph under the full constraints did not finish within 30 minutes of MaxSAT. Whether the published vertex counts survive is therefore open.

7.2 Stored circuits that do not prepare the CAT state

We simulated all 568 stored circuits (`spidercat/circuits`) without noise in stim's tableau simulator. 550 prepare the CAT state on qubits $0, \ldots, n - 1$. The other 18 do not:

- the $t = 7$ circuits for $n \in \{37, 38, 41, 42, 43, 46, 47, 48\}$;
- all 10 stored $t = \infty$ circuits `cat_state_tinf_n*`, for $n \in \{10, 17, 20, 22, 25, 32, 35, 40, 47, 50\}$.

In each of these, the GHZ state extends to between 1 and 30 qubits beyond the data range, and those qubits are never measured.

The cause is the extractor defect of Section 4.5:

- the stored spanning trees of these $t = 7$ graphs leave 1–2 unmarked co-tree edges;
- every prime-inverse graph has $n + 1$ co-tree edges but only n marks.

7.3 The $t = \infty$ branch of `generate.py`

`cat_state_FT(n, math.inf)` uses `construct_prime_inverse_graph(2n+3)`: a cycle with chords $v \leftrightarrow v^{-1} \bmod p$, with vertex ratio 2. It returns `{}` unless $p = 2n + 3$ is an admissible prime.

- Among accepted n that reach this branch, the graph is exactly robust only for $n \in \{7, 10, 17\}$. ($n = 2$ and 5 are robust too, but are handled by other code paths.)
- Every accepted n with $20 \leq n \leq 67$ fails (Table 11). Heuristic lower bounds on the excess grow from 2 at $n = 20$ to 15–16 at $n = 62$–67.
- For every admissible p the n marks form a perfect matching. Contracting it gives a loopless 4-regular multigraph on n vertices with a few parallel edges. Monien and Preis bound the bisection width of large 4-regular graphs by $(2/5 + \varepsilon)|V|$ [6], which is below the $\lfloor n/2 \rfloor$ that $t = \infty$ requires. Their result is stated for simple graphs, so the parallel edges need a small adjustment.
- The 10 stored $t = \infty$ circuits come from this branch. None of them prepares the CAT state (Section 7.2), and for $n \geq 20$ the underlying graphs are not robust either.

7.4 The verifier is unsound

`find_marking_property_violation` has three defects:

- **Balanced cuts.** For a vertex set S the adversary can split the marks of cut edges arbitrarily, which gives $\min(M_A + M_{\text{cut}}, M_B + M_{\text{cut}}, \lfloor n/2 \rfloor)$. The code instead uses $n - \min(\cdot) = \max(M_A, M_B)$ whenever $\min(\cdot) > n/2$ (line 73).
- **Isolated marks.** It never isolates a block of ≥ 3 marks inside one edge.

Table 11: Prime-inverse graphs used by the $t = \infty$ branch. Every violation witness was recomputed from scratch on G'. "search" rows give heuristic witnesses (lower bounds on the excess).

p	n	V	result	method	witness
7	2	4	robust	exhaustive	–
13	5	10	robust	exhaustive	–
17	7	14	robust	exhaustive	–
23	10	20	robust	exhaustive	–
37	17	34	robust	MILP	–
43	20	40	violated	search	cut 8, marks $10 \mid 10$
47	22	44	violated	search	cut 8, marks $10 \mid 12$
53	25	50	violated (excess 2)	MILP	–
67	32	64	violated	search	cut 12, marks $16 \mid 16$
73	35	70	violated	search	cut 8, marks $14 \mid 21$
83	40	80	violated	search	cut 14, marks $20 \mid 20$
97	47	94	violated	search	cut 12, marks $23 \mid 24$
103	50	100	violated	search	cut 14, marks $24 \mid 26$
107	52	104	violated	search	cut 18, marks $26 \mid 26$
113	55	110	violated	search	cut 16, marks $27 \mid 28$
127	62	124	violated	search	cut 12, marks $28 \mid 34$
137	67	134	violated	search	cut 16, marks $36 \mid 31$

- **Node labels.** It indexes `marks_adj` by node labels, so labels must lie in $[0, |E|)$.

Two counterexamples, on which the function returns no violation (both re-checked exactly):

- the triangular prism with all 9 edges marked, which has a weight-3 cut with marks $4 \mid 5$;
- the smallest found: K_4 with marks $(0,2)$:1, $(0,3)$:2, $(1,2)$:2, $(1,3)$:1, $(2,3)$:2 ($n = 8$), which has a weight-3 cut with marks $4 \mid 4$.

This unsoundness did not cause the $t = 7$ problem, which the verifier does detect. It matters for balanced cuts and multi-mark edges.

7.5 Corrections to the paper

- Definition 5.3: "at most t marks" should read "at most w marks" for a cut of w edges.
- Proposition 5.3 equates "t-FE" with t-robust; by Definition 2.4 it should be "t-FT" (that is, $(t+1)$-FE).
- Appendix E.2: the $T = 7$ encoding lacks the subtree constraints, and the general subtree bound should be $|S| + 2$, not t.
- The paragraph after Theorem 5.5 says consecutive Morgenstern graph sizes differ by $O(1)$. They grow geometrically; the proof of Corollary 5.1 correctly uses only a bounded ratio.

8 Recommendations

1. **Do not implement OpenAI's construction.** If an expander starting graph is needed, sample a random one and certify it with the (one-sided) final test.
2. $n \leq 17$**:** use SpiderCat's stored graphs at $t = \lfloor n/2 \rfloor - 1$ (verified exactly for $n \leq 13$) and the new 14–17-terminal graphs.
3. $18 \leq n \leq 100$**:** use the dense route. Vertex ratio is 3.7–7.6, the certificate is fully exact and takes milliseconds to seconds, and circuits have $V + n + 1$ CNOTs. Against the Ramanujan route this saves a factor of about 2.6 in CNOTs at $n = 24$ and 1.5–1.9 at $n = 50$–100.

4. $n > 100$**:** use the Ramanujan route (vertex ratio 12, certified in seconds up to $n \approx 2 \times 10^4$); the variant $d = 11$, $k = 2$ (ratio 12.5) gives spectral slack. Beyond that, the dense $O(N^3)$ certificate becomes the bottleneck. Extracting circuits for $V \gtrsim 900$ needs a faster spider-ordering-tree heuristic than the MILP.

5. **Repository:**

   - add the subtree constraints for $|S| \leq 5$ to the $t = 7$ marking, fix the range in `general_solver`, and regenerate the $t = 7$ graphs and circuits;
   - fix the extractor's handling of unmarked co-tree edges, or use the new flag-aware extraction (`extract_circuit_sparse`), and regenerate the circuits of Section 7.2;
   - remove the prime-inverse $t = \infty$ branch and its stored circuits;
   - replace the verifier by an exact check (the cut-bounded MILP took a median of 1.1 s per stored graph, at most 8.1 s, and 12.9 min for all 437);
   - turn on verification by default.

A Reproducibility

Code. Commit `2849c48` on branch `openAI-spider` (on top of `main` at `f239d12`) adds the files below together with `uv.lock`; no existing file was modified. This report is in `reports/` on the same branch.

- `spidercat/graphs_ramanujan.py`:
  - `ramanujan_tinf_marked_graph` and `dense_tinf_marked_graph` (generation);
  - `verify_composition_certificate` (independent certificate);
  - `max_margin_milp` and `max_margin_exhaustive` (exact checks);
  - `max_margin_exhaustive_subdivided` (exact check over all of $V(G')$, for cross-checking);
  - `lambda2_certificate` (rigorous spectral test);
  - `spider_ordering_tree_milp` and `extract_circuit_sparse` (circuits);
  - `circuit_fault_check` (circuit-level faults of weight ≤ 2).
- `spidercat/tinf_gadgets/gadget_T{14..17}.json`: the new gadgets.

Command line: `python -m spidercat.graphs_ramanujan N --route dense|ramanujan --out graph.json`. Output files store `G.edges` and `M_inv` (as in `circuits_data`), plus `n`, `t`, `route`, `N`, `V`, `cluster_of` and `certified`. The certificate can be re-checked from the file alone. These files contain no spanning tree or matching; `spider_ordering_tree_milp` computes them.

Environment. Python 3.12.5, NumPy 2.4.4, SciPy 1.16.2 (HiGHS), NetworkX 3.5, python-sat 1.9.dev2, stim 1.15.0; Apple M3 Pro, 11 cores, 36 GB.

Sources. The OpenAI preprint: `https://github.com/openai/math/tree/main/preprints/Deterministic-nonbipartite-Ramanujan-graphs-in-every-fixed-degree-September-23-2026`. SpiderCat: arXiv:2603.05391v1.

References

[1] A. B. Khesin, S. M. Li, B. Poór, B. Rodatz, J. van de Wetering, R. Yeung, *SpiderCat: Optimal fault-tolerant cat state preparation*, arXiv:2603.05391 (2026).

[2] OpenAI, *Deterministic nonbipartite Ramanujan graphs in every fixed degree*, OpenAI Math Release preprint, 23 September 2026.

[3] A. W. Marcus, D. A. Spielman, N. Srivastava, *Interlacing families IV: bipartite Ramanujan graphs of all sizes*, SIAM J. Comput. 47 (2018).

[4] M. B. Cohen, *Ramanujan graphs in polynomial time*, FOCS 2016.

[5] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM (2002), Theorem 10.3.

[6] B. Monien, R. Preis, *Upper bounds on the bisection width of 3- and 4-regular graphs*, J. Discrete Algorithms 4 (2006).

[7] C. Gidney, *Stim: a fast stabilizer circuit simulator*, Quantum 5, 497 (2021).